TiltTwin

Remote CAD teaching through physical motion · Design process write-up

CPSC 581 Fall 2026 · Group Project 2: Expertise Sharing · **[Team member names]** · Source: github.com/Kevin398/CPSC581-TiltTwin · Live: tilttwin.vercel.app

> **In one sentence.** An expert on a laptop demonstrates a rotation of a CAD part. A beginner on a phone reproduces it by *physically turning the phone*, so the virtual part turns as if it were held in their hand. Both people see live angle, direction and colour feedback, and both get a ✓ when the beginner matches.
> **Expertise conveyed:** spatial understanding of 3D rotation, meaning which axis, which direction and how far. Novices in Blender or Fusion struggle with this because the mouse mapping (orbit vs. rotate, gimbal axes) is abstract.
> **Configuration:** one expert (laptop), one novice (smartphone), synchronous and remote, joined by a 4-letter room code.

---

1 · Process overview

We alternated divergent and convergent phases over five iterations. Each iteration produced an artifact (sketch set, storyboard, wireframe or prototype) that was critiqued before the next step.

① **Diverge**	② **Converge**	③ **Refine**	④ **Specify**	⑤ **Prototype**	⑥ **Evaluate**	⑦ **Extend**	⑧ **Polish**
10 interaction concepts sketched (Fig. 1)	Compare against criteria; pivot from CADSense to TiltTwin (Table 1)	5 refinement sketches; drop transparency (Fig. 2)	Storyboard; wireframes v1 → v2 (Figs. 3–4)	v1: mouse baseline (Fig. 5); v2: phone tilt + sync (Figs. 6–8)	Technical checks; real-phone test; A/B user test (§9)	Real assembly tasks, hot/cold sound and haptics (§7)	UI/UX audit, fixed join link (§8)

Methods and tools: individual rapid sketching (in-class "10 ideas" technique), team critique in Discord, a criteria-based comparison matrix, a storyboard, low-fidelity wireframes in two versions, an interactive high-fidelity prototype (React + Three.js), automated unit tests of the sensor mapping, and a within-subjects A/B usability test comparing mouse input with phone-tilt input.

---

2 · Divergence: ten interaction concepts

Each concept answers the same question in a different way: *how can an expert's knowledge of 3D orientation reach a novice through the body rather than through words or a shared screen?*

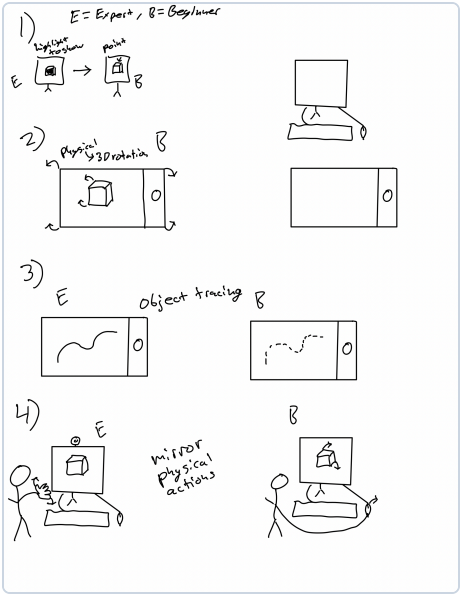


Fig. 1a Concepts 1–4: highlight and point; physical phone rotation; path tracing; mirroring the expert's hand motion (webcam).

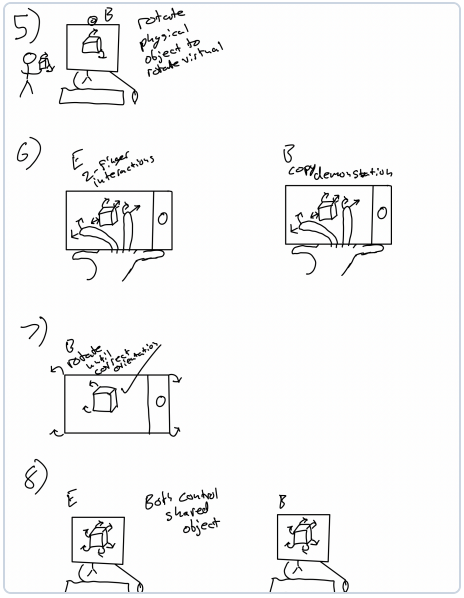


Fig. 1b Concepts 5–8: rotate a physical proxy object; copy two-finger gestures; rotate until correct; both control one shared object.

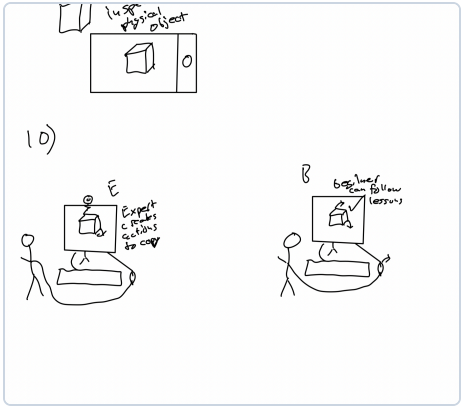


Fig. 1c Concepts 9–10: inspect a physical object beside its virtual twin; the expert records actions as lessons the beginner follows.

3 · Convergence: choosing the concept

Our first direction, **CADSense**, was a mouse-driven tutor covering extrude, rotate and scale. The beginner followed on-screen arrows to match a target model. When we checked it against the brief and rubric (25% of the grade is creative *embodied* input and output, and video calls or screen sharing as the core interaction are explicitly discouraged), CADSense's mouse input looked conventional. We scored the strongest concepts from Fig. 1 against criteria taken directly from the brief.

Concept	Embodied input	Embodied output	Conveys CAD expertise	Feasible in ~2 weeks	Reliable live demo	Total
CADSense (mouse + arrows, 3 operations)	1	2	3	2	3	11
#3 Object tracing (draw a path)	2	1	1	3	3	10
#4 Mirror hand motion (webcam tracking)	3	2	2	1	1	9
#5 Rotate a physical proxy object	3	2	3	1	1	10
#8 Both control a shared object	2	2	2	2	2	10
#2 + #7 + #10 → TiltTwin (phone is the part)	**3**	**3**	**3**	**2**	**2**	**13**

Table 1. Scores from 1 (weak) to 3 (strong). **[Team: confirm or adjust scores from your discussion.]** The proxy-object idea (#5) scored high on embodiment, but it needs tracking hardware or computer vision we could not make reliable in time. A

phone already carries an orientation sensor, so **the phone itself becomes the physical proxy**.

> **Decision.** Merge #2 (physical rotation), #7 (rotate until correct) and #10 (expert creates actions to copy) into TiltTwin. We deliberately **gave up breadth** (extrude and scale) to make one embodied interaction, rotation, complete and polished. This also gives a clean A/B comparison with mouse rotation (§9).

---

4 · Iteration 1: refining TiltTwin

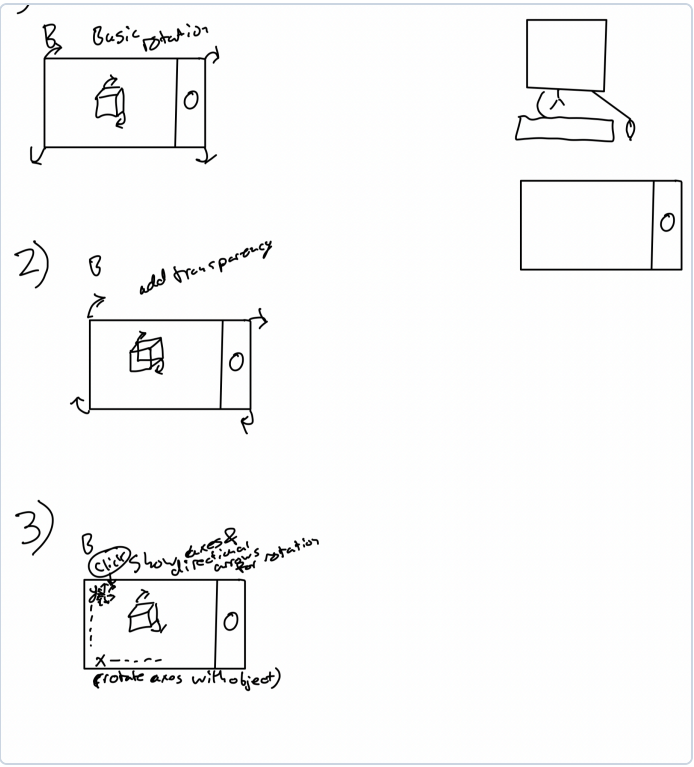


Fig. 2a (1) basic physical rotation; (2) a transparent "ghost" of the target over the part; (3) tap to show axes and direction arrows that rotate with the object.

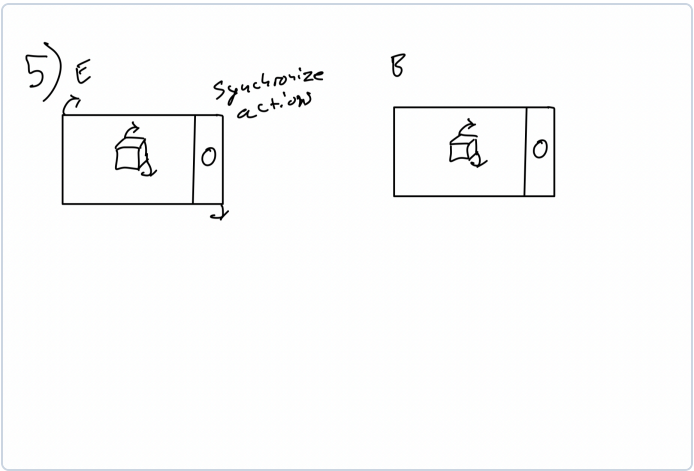


Fig. 2b (4) ✓ feedback on reaching the correct orientation; (5) synchronize the expert's and beginner's actions.

Kept: 1, 3, 4, 5. **Dropped:** 2, the transparency overlay. On a phone-sized screen, two overlapping, semi-transparent L-brackets are hard to read. It becomes unclear which edges belong to which part, and when the part is close to the target the two shapes merge. The overlay also invites "matching shapes by eye" rather than understanding *which axis and which direction*, which is the expertise we want to convey. We replaced it with three cues that each answer one question:

- **Where should it end up?** A small window showing the expert's *demonstration*: the part animating from 0° to the target on a loop.
- **Which way do I turn?** A coloured arc arrow around the highlighted axis, from the current angle to the target.
- **How far am I?** A large live gauge ("22° / 45°") plus colour coding: orange when far, yellow when close, green when matched.

[Team: add any extra reason from your own discussion of transparency.]

5 · Iteration 2: storyboard and wireframes

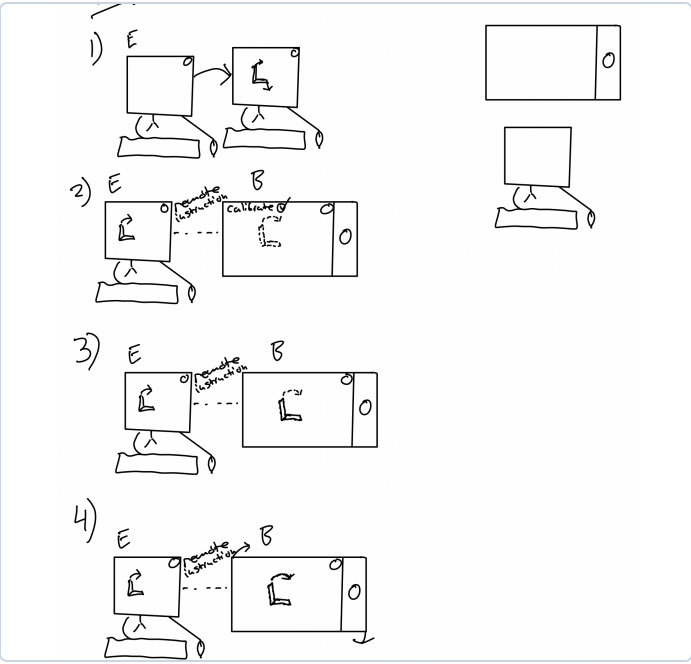


Fig. 3 Storyboard. The expert sends a remote instruction (1–2); the beginner calibrates (2); the beginner rotates the phone and the part follows (3–4); both see the completion check (5).

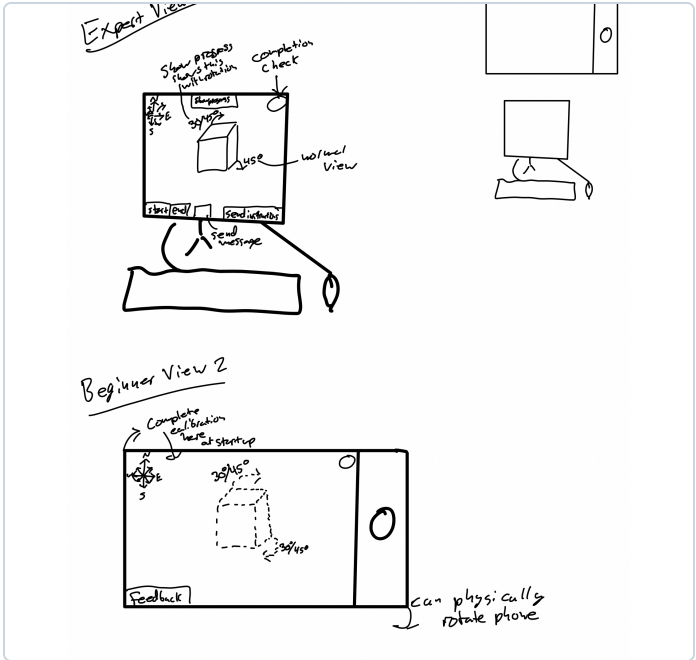


Fig. 4 Wireframes v2. The expert sees the beginner's progress as an angle ("30°/45°") on the model, plus start/end, send instruction, send message and a completion check. The beginner calibrates at startup, sees the same angle labels, and physically rotates the phone.

v1 → v2 changes (v1 is in `docs/design/sketches/04-…`): the progress indicator changed from a vague "show progress" button to a **live angle on the model itself**, shown to *both* people. Showing "30°/45°" to both gives the expert a precise thing to correct ("15° more") and gives the novice an exact goal. v2 also makes **physically rotating the phone** the explicit input, replacing on-screen controls.

6 · Iteration 3: prototypes

Prototype v1: mouse baseline (CADSense-style, "Version A")

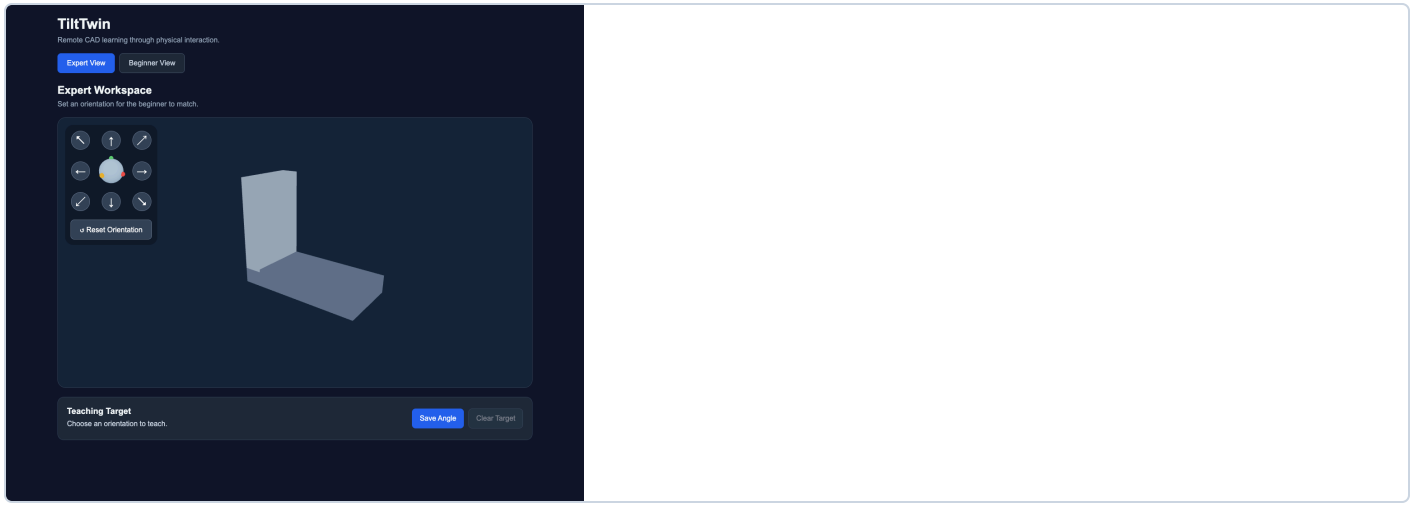


Fig. 5 v1 expert workspace. Free 2-axis rotation by mouse drag or an 8-way arrow pad, plus "Save Angle". The beginner pressed "Check Angle" for feedback. Both roles ran in one browser tab.

What v1 taught us. Free 2-axis rotation made instructions hard to express ("rotate it like… this"). Feedback only came when the beginner pressed a button. Running in one tab meant there was no remote collaboration yet. These became the requirements for v2: **one axis per lesson**, **continuous feedback** and **real two-device sync**.

Prototype v2: TiltTwin (final system)

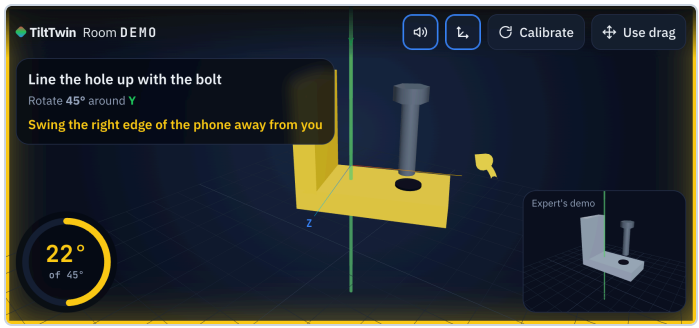


Fig. 6a Beginner, mid-rotation. The instruction gives a body-centred hint ("Swing the right edge of the phone away from you"). The Y axis is highlighted, the arc arrow shows the remaining turn, the gauge reads 22°/45°, and everything is yellow because the part is close. The expert's looping demo plays bottom-right.

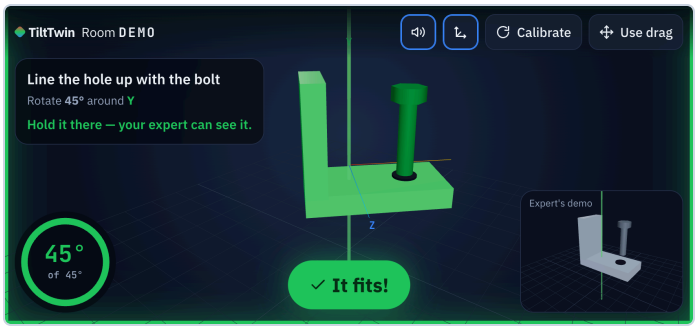


Fig. 6b Held within 8° of the target for 0.8 s. The part, frame and gauge turn green, "✓ Matched!" pops in, and the phone chimes and vibrates (multi-sensory output).

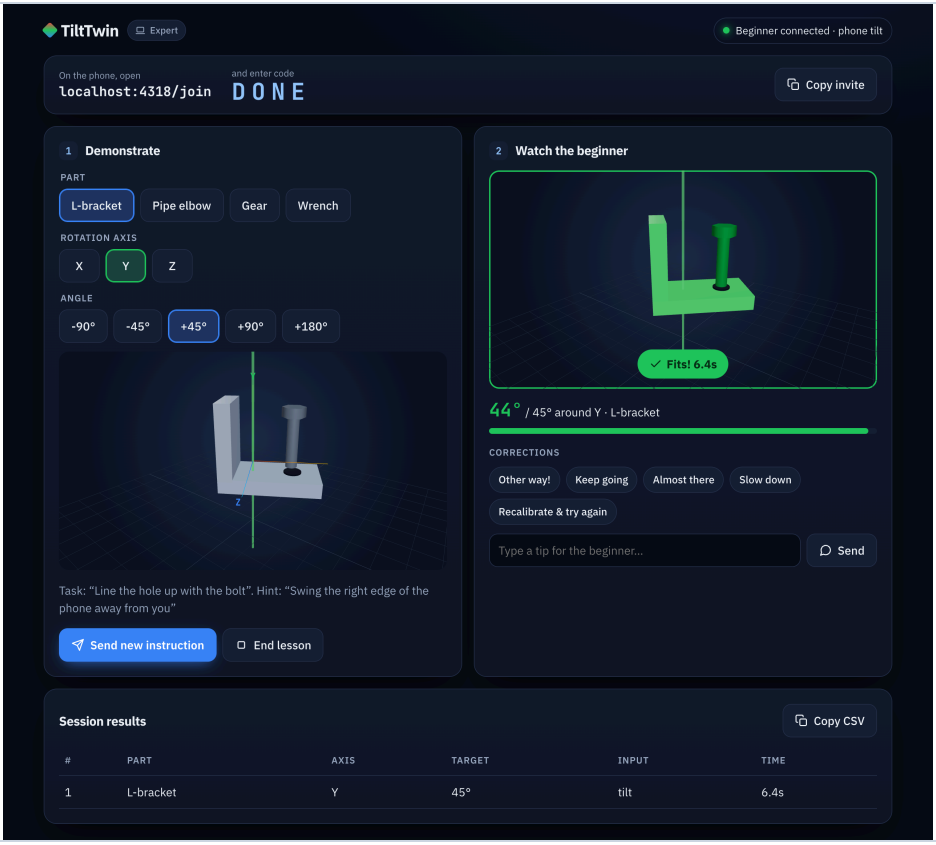


Fig. 7 Expert view. Left: choose an axis and angle, preview the demonstration loop, and send. Right: a live mirror of the beginner's part, angle readout, quick corrections and a free-text tip. Bottom: the session log records each lesson's input type and completion time (also used for our user test).

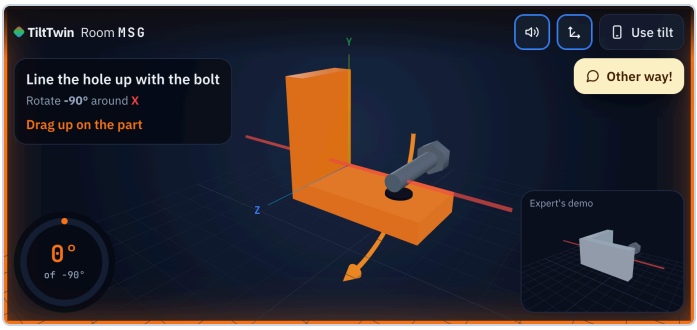


Fig. 8a The expert's correction ("Other way!") arrives as a toast. A –90° X lesson shows the arc arrow for the full remaining turn.

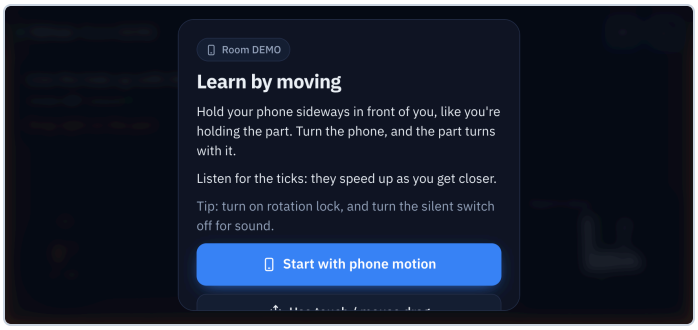


Fig. 8b Onboarding: how to hold the phone, a rotation-lock tip, and a drag fallback for devices without motion sensors.

Key design decisions in v2

Decision	Why	Alternatives considered
The phone is the part. Turning the phone turns the part the same way, measured around the lesson's axis.	The most direct embodied mapping: the hand learns the rotation, not just the eye.	On-screen arrows (v1); full free 3-DoF rotation (too hard to judge against one target).
One axis per lesson (swing-twist decomposition isolates rotation around X, Y or Z).	Hands wobble. Ignoring off-axis wobble keeps feedback stable and matches how CAD tools teach rotation about one axis.	Matching full orientation (frustrating jitter); Euler angles (gimbal lock near an upright phone).
Calibration: the current pose becomes 0° when a lesson arrives, and a Calibrate button is always available.	Everyone holds the phone differently. A relative zero avoids compass drift and removes "hold it exactly like this" instructions.	Absolute compass heading (drifts indoors; differs between phones).
Body-centred hints ("Tilt the top toward you", "Swing the right edge away from you").	Translates CAD axis language into motor language. Signs are verified by unit tests against the sensor specification.	Showing only X/Y/Z and ± (abstract for novices).
8° tolerance, 0.8 s hold.	Sweeping past the target shouldn't count. 8° is tight enough to be meaningful but achievable by hand.	A "Check Angle" button (v1, delayed feedback); 5° (too strict by hand). **[Tune after testing]**
Feedback in four channels: colour, number, arrow, sound/vibration.	The beginner's eyes move between the part and the phone's orientation, so redundant cues help. Vibration confirms success without looking.	Transparency overlay (dropped, §4).
Quick corrections plus free text for the expert.	The expert watches a live mirror and must correct within seconds; one-tap phrases are faster than typing.	Voice or video call (discouraged by the brief; doesn't use the shared model).
Firebase Realtime Database rooms with a 4-letter code.	Sub-second sync without our own server. Codes avoid confusable characters (no 0/O, 1/I).	WebRTC peer-to-peer (fails behind some NATs; risky for the live demo).

---

7 · Iteration 4: from "match an angle" to "make it fit"

Once the core interaction worked on real phones, we asked how to make it **more useful in real CAD work** and **more engaging**, without weakening the embodied core. We listed candidate additions and chose the ones that strengthen embodied output or real-world meaning while staying cheap enough to build reliably before the deadline:

Candidate	Value	Cost or risk	Decision
Part library	Beginners practise on parts they will actually model	Low	**Built**
Assembly "fit" task with a mating part	Gives the rotation a purpose: in CAD assemblies you orient a part so it mates with another	Medium	**Built**
Hot/cold vibration	Feel the answer without looking: strong embodied output	iPhone Safari has no vibration API	**Built, with sound as the guaranteed channel**
Stars and streaks	Fun and motivating	Low, but weak link to expertise sharing	Deferred

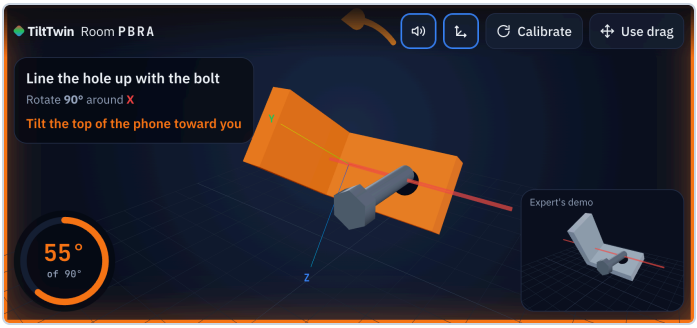


Fig. 9a L-bracket: line its hole up with the bolt (X axis).



Fig. 9b Pipe elbow: connect its outlet to the pipe (Y axis).

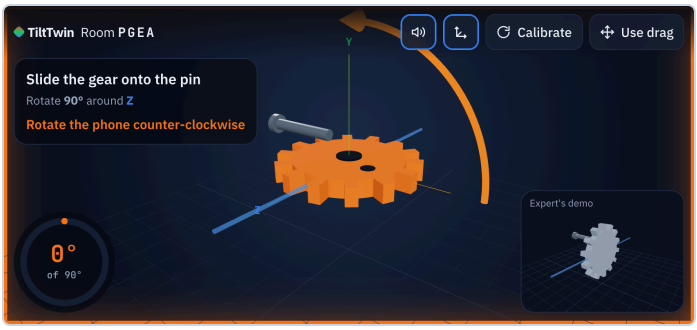


Fig. 9c Gear: slide its pin hole onto the pin (Z axis).

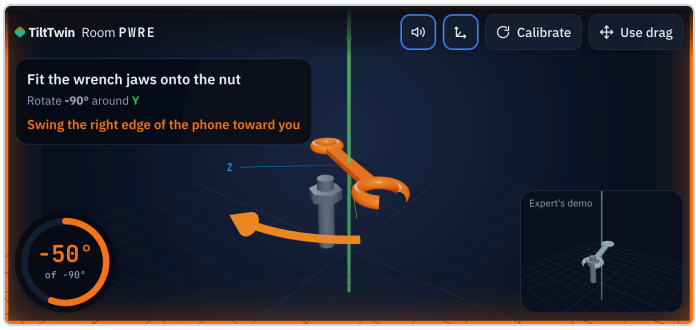


Fig. 9d Wrench: fit the jaws onto the nut (–90° about Y). The arrow shows a clockwise turn.

- **Mating parts, not a ghost.** Each part has a *feature* (a hole, an outlet or the jaws), and a solid mating part (a bolt, pipe, pin or nut) is fixed where that feature will be at the target. This keeps the decision from §4: no transparent overlay. It also turns an abstract angle into a physical goal, and the moment of success becomes "it clicks into place". A unit test checks that no feature sits on an X/Y/Z axis line; otherwise rotating about that axis would never visibly move it, and the task would be impossible to see.
- **Hot/cold guidance.** While a lesson is active, the phone ticks. The ticks speed up (from ~1 to ~9 per second) and rise in pitch as the part gets closer, and stop when it fits. On Android each tick is also a vibration pulse. On iPhone we use the light haptic that iOS 18+ gives when a native switch control is toggled, because Safari has no vibration API. That trick is undocumented, so **sound is the guaranteed channel**. The beginner can find the target by ear and by feel while their eyes are on the part. A 🔊 button turns it off.
- **Expert side.** The expert chooses the part as well as the axis and angle. The demonstration loop and the live mirror both show the mating part. The session log records the part, so the A/B test can cover different parts.

8 · Iteration 5: usability and visual audit

Before the critique we audited the whole interface against a professional UI/UX checklist (the *UI/UX Pro Max* guideline database: accessibility, touch, typography, motion and forms). We also simplified how beginners join, after noticing that a new long link per lesson is awkward to send to a phone.

Finding	Guideline	Change
A new long link for every lesson had to be sent to the phone	Predictable navigation; fewer steps	One fixed link (`tilttwin.vercel.app/join`, easy to bookmark) plus a 4-letter code shown large on the expert's screen. The code is checked against live rooms, and a mistyped code gets an inline error.
Emoji used as icons (🔊 💬)	One consistent SVG icon family	Inline stroke icon set; icon-only buttons have accessible names
No visible keyboard focus on dark buttons	Focus states (high severity)	`:focus-visible` ring on every control
Phone toolbar buttons were 38 px	Touch targets ≥ 44 px	All controls ≥ 44 px
Code field used a placeholder as its label	Visible labels; errors next to the field	Visible label, helper text and inline error
Thin bar meter for progress	Glanceable feedback	Radial progress ring around the live angle, in monospaced numbers that don't jitter
Home page didn't show the product	Show the core interaction first	Interactive 3D hero (pick a part, watch it fit) and a three-step "How it works"
Success was a static badge	Motion should convey meaning	A pulse ring for the "click into place" moment; all motion is disabled under reduced-motion

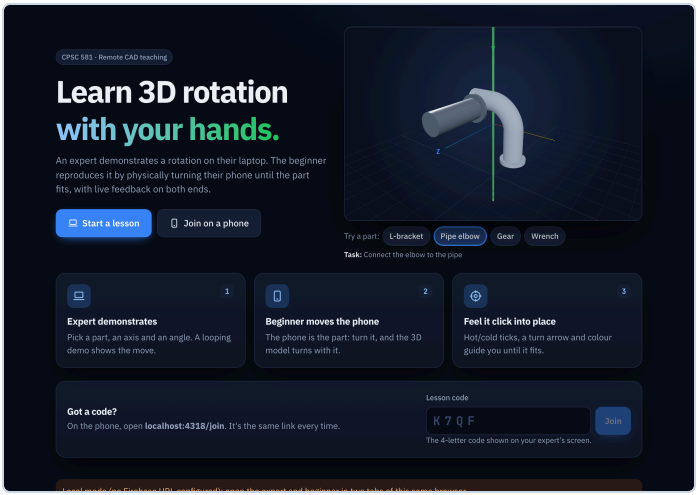


Fig. 10a Home: interactive 3D preview with part chips, "How it works", and code entry.

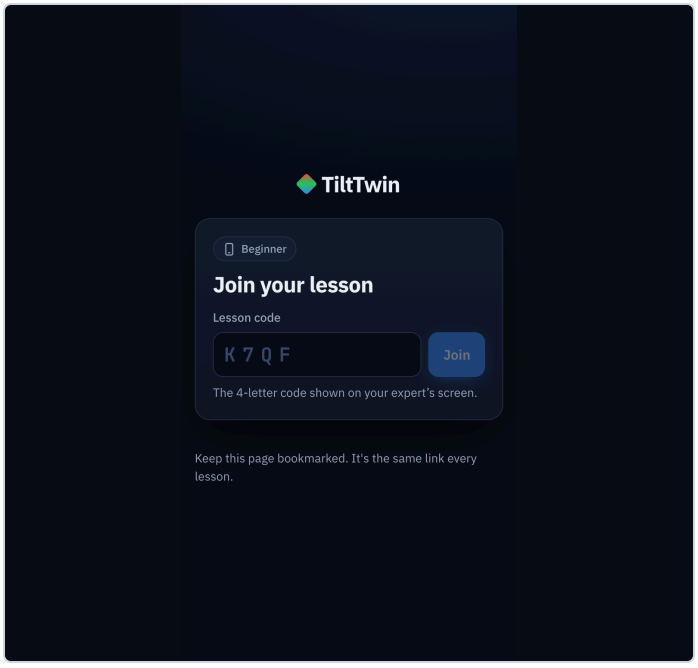


Fig. 10b The fixed join page on a phone. Typography is IBM Plex Sans with JetBrains Mono for codes and angles.

9 · Evaluation

9.1 Technical verification (done)

- **Sensor mapping unit tests** (`npm test`, 6 tests pass): wrap-around at ±180°; twist extraction ignores off-axis wobble; and for portrait and both landscape holds, the physical motion named by each hint produces a positive angle on its axis. The test builds realistic phone poses and converts them to the angles a browser reports. One test initially failed: our assumption about the sign of `beta` was wrong. We corrected the test against the W3C definition, which confirmed the on-screen hints were right.
- **Simulated-sensor runs**: synthetic `deviceorientation` events drove the beginner view through far → close → matched, and the expert view mirrored each state in a second tab.
- **Bug found and fixed**: the "matched" state never fired, because each sync update created a new lesson object and restarted the 0.8 s hold timer. We fixed it by keying the timer on the lesson id.
- **Real devices:** **[Team: list the phones and browsers tested, e.g. iPhone 14 / Safari, Pixel 7 / Chrome, and any issues.]**

9.2 Usability test: mouse (A) vs. phone tilt (B)

Protocol. Within-subjects, counterbalanced (half start with A). Each participant gets three lessons per condition (e.g. bracket Y +45°, elbow X –90°, gear Z +90°) from a team member acting as the expert. The session log records completion time; we note wrong-direction starts, expert corrections sent, and a 1–5 rating for "I understood which way to rotate". Condition A is TiltTwin's **drag mode**, so the only difference is the input modality. Participants: **[n = ?, who]**.

Measure	A · Mouse/touch drag	B · Phone tilt	Notes
Mean completion time (s)	**[ ]**	**[ ]**	
Wrong-direction starts (total)	**[ ]**	**[ ]**	
Expert corrections sent	**[ ]**	**[ ]**	
"Understood the axis" (1–5, mean)	**[ ]**	**[ ]**	
Preferred condition (count)	**[ ]**	**[ ]**	

Observations and quotes: **[What surprised you? Where did people hesitate? Any quotes?]**

Changes made because of testing:

- **Y-axis hint reworded.** In our first real-phone test, X and Z lessons were completed as instructed, but for Y the tester moved the opposite way. The original hint, "Turn the phone to the left", can be read two ways: the phone turning left (its screen ends up facing right) or the screen turning to face left. We checked the sensor maths against the W3C angle definitions for portrait and both landscape holds (now automated tests). The rotation itself was correct, so the ambiguity was in the words. The hint now names a specific edge: "Swing the right edge of the phone away from you".
- **[Add further changes from the A/B session.]**

10 · Known limitations and incomplete functionality

- **One axis per lesson** (by design). Compound rotations would be a sequence of lessons.
- **Rotation only**: no extrusion or scaling, and no import of real Blender or Fusion files. There are four predefined parts (bracket, pipe elbow, gear, wrench).
- **iPhone haptics** rely on an undocumented iOS 18+ behaviour and may not fire on older iOS. Hot/cold **sound** always works, but is muted when the iPhone's silent switch is on.
- **One expert and one beginner per room.** A second beginner would overwrite the first one's progress.
- **Y-axis drift**: turning left/right relies on the gyroscope and compass and can drift slowly on some phones. Press Calibrate to fix it.
- **Screen auto-rotation**: large Z-axis turns can make the phone flip its layout unless rotation lock is on. The mapping stays correct, but the HUD jumps.
- **Security**: rooms are open to anyone with the code, and the database rules only validate shape. This is acceptable for a class demo, not for production.
- The expert's session log lives in that browser tab and is lost on reload; use "Copy CSV" to keep it.

11 · Credits and AI use

Sketches, storyboard and wireframes: **[names]**. Baseline prototype: **[names]**. Built with React, TypeScript, Three.js (react-three-fiber, drei), Firebase Realtime Database and Vite. **AI assistance:** Claude (Anthropic) was used to implement parts of the v2 prototype (phone-orientation hook, sync layer, feedback UI), write the unit tests, generate screenshots, and draft this document. All design decisions were made and reviewed by the team. **[Team: adjust to match your actual use.]**